

Full Project Audit

Architecture, application flow, database design, operational state, and prioritized recommendations.

Prepared 17 August 2026

Scope: source-code and local-runtime review; not an external penetration test.

Audit outcome

A well-scoped teaching/demo incident-management application. The core MERN workflow is coherent and builds successfully. Production deployment requires security, configuration, data-safety, testing, and operations work first.

1. Executive Summary

AutoReliability / IncidentOps is a MERN-based incident-management and on-call tracker. It is designed as a PagerDuty-style teaching project: teams can log in, create and track incidents, assign engineers, manage incident status, keep a timeline, write postmortems, create in-app notifications, and export a postmortem PDF.

Overall assessment: The application is a strong classroom/demo foundation with understandable React, Express, and MongoDB separation. It should not yet be treated as production-ready because several defaults and missing controls create operational and security risk.

What was audited

- Frontend: React/Vite application, routing, authentication state, API client, screens, user interaction, and styles.
- Backend: Express bootstrap, routes, controllers, authorization, error handling, background reminder job, and API behavior.
- Data layer: MongoDB/Mongoose schemas, relationships, seed behavior, notification persistence, and calculated MTTR.
- Operations: environment configuration, scripts, installer, documentation, build and test coverage.

Observed local runtime state

- The API health endpoint on port 5001 returned success during the review.
- Final verification authenticated the documented demo admin successfully. The local database then contained 4 users, 3 incidents (1 open, 1 investigating, 1 resolved), 2 on-call users, and no notifications.
- The seeded resolved incident produces an average MTTR of 0 minutes because it is resolved immediately in the seed data; this is expected demo data, not a meaningful operational metric.
- The database state changed during the review: an earlier diagnostic found no User records, while the final check found seeded demo data. This confirms the seed/reset state is operationally significant and should be controlled deliberately.
- The frontend production build completed successfully with Vite.

Important: The seed command deliberately deletes Users, Incidents, and Notifications before rebuilding sample data. Treat it as a local-demo reset, never as an unguarded production remedy.

2. Purpose and Product Scope

The project teaches the incident lifecycle: detect an issue, create an incident, assign ownership, investigate through timeline updates, resolve it, document the root cause and corrective actions, then review metrics such as MTTR. Its UI and seeded examples intentionally resemble a simplified reliability command center.

Primary users

- Administrators: can create and remove user accounts, manage incidents, and delete incidents.
- Engineers: can view the incident board, create/update incidents, add comments, receive in-app notifications, and contribute to postmortems.

Key user outcomes

- A shared Kanban view of open, investigating, and resolved incidents.
- Clear incident ownership, severity, impact, timeline, and resolution context.
- Basic on-call roster visibility and notification reminders.

3. Architecture at a Glance

```
Browser
  React + Vite + React Router
  AuthContext stores JWT in localStorage
  Axios sends Authorization: Bearer <token>
    |
    v
Express API (port 5001)
  auth / incidents / notifications routes
  JWT middleware + role checks
    |
    v
MongoDB via Mongoose
  Users <-> Incidents <-> Notifications
    ^
    |
  reminderJob periodically creates in-app notifications
```

The frontend and backend are deployed/run separately. The frontend defaults to `http://localhost:5001/api` when `VITE_API_URL` is not set. The backend defaults to a local MongoDB database named `reliability_command_center` when `MONGODB_URI` is not set.

4. Frontend Review

Composition and routing

- `main.jsx` mounts React in StrictMode, `BrowserRouter`, `AuthProvider`, and `ToastProvider`.
- `App.jsx` protects the dashboard, incident detail, and team routes through `PrivateRoute`; unauthenticated users are redirected to `/login`.
- `Layout.jsx` supplies the product chrome, navigation, logged-in identity, and notification menu around routed content.

Screen behavior

- Login: posts email/password to /auth/login; saves the returned token and user state.
- Dashboard: loads incidents, statistics, and users; provides search, a three-column status board, drag/drop status updates, and creation modal.
- Incident detail: supports edits, status and assignment changes, timeline comments, @mention selection, postmortem fields, admin deletion, and client-side PDF export.
- Team: lists on-call personnel and exposes admin-only create/delete user workflows.

Frontend data flow

api.js creates one Axios client. An interceptor reads rcc_token from localStorage and attaches it as a bearer token. AuthContext restores a previous session by calling /auth/me. Page components issue REST calls directly and reload collections after mutations.

Strengths

- Small, readable component structure with a clear service boundary for API calls.
- Route-level authentication and role-aware destructive controls are visible in the UI.
- Toast feedback and custom confirmation modals improve the interaction model over browser alerts.
- Global typography is now centralized: Manrope for normal UI text and Syne for headings.

Frontend gaps to address

- Most data-loading calls have no loading/error state. A failed request can leave an empty or indefinitely loading view.
- The JWT is stored in localStorage, which makes it accessible to any injected script. For a production web app, favor secure httpOnly cookies plus CSRF protections or maintain robust XSS controls.
- Drag/drop and popup/modal interactions need stronger keyboard support, focus management, and semantic button/list markup for accessibility.
- The user-creation password input is visibly rendered as text and defaults to a weak demo password.

5. Backend and API Review

Startup and request handling

server.js loads environment variables, configures CORS, JSON parsing, request logging, route groups, a health endpoint, centralized error handling, MongoDB connectivity, and the reminder scheduler. The server starts listening only after MongoDB connects.

Authentication and authorization

- POST /api/auth/login checks email/password with bcrypt and returns a signed JWT plus a public user object.
- protect middleware verifies the bearer token, retrieves the current user from MongoDB, and attaches it to req.user.
- adminOnly uses the database-backed req.user role, not just the role stored in the token. This is a good authorization choice.

API surface

- Auth: login, current-user lookup, user list, admin user creation, and admin user deletion.
- Incidents: list/filter, statistics, create, detail, update, delete (admin), and timeline comments.
- Notifications: current-user/global list and mark-as-read.

Workflow details

- Creating an incident sets reportedBy from the authenticated user and adds a created timeline entry.
- Changing status adds a status event; resolving an incident sets resolvedAt, which feeds the MTTR virtual/calculation.
- Changing assignment sends an in-app notification to the assignee.
- Comments parse @mentions, look up matching users, and create notifications for them.
- The reminder job finds stale high/critical unresolved incidents and adds a notification plus a system timeline event.

Backend strengths

- Controller/route/model separation is clear and appropriate for the project size.
- Passwords are hashed with bcrypt and are excluded from ordinary User queries.
- Mongoose enums constrain role, severity, status, timeline type, and notification channel values.
- A simple health endpoint supports basic liveness checks.

6. Database Design and Behavior

Collections

- User: name, normalized unique email, bcrypt password hash, role, team, and on-call state.
- Incident: title, service, description, severity, status, assignedTo, reportedBy, timestamps, postmortem fields, action items, and embedded timeline.
- Notification: title, message, delivery channel, target user, optional incident reference, read state, and timestamps.

Relationships

```
User 1 --- reports ---> many Incidents
User 1 --- assigned ---> many Incidents
User 1 --- authors ---> embedded Incident timeline events
User 1 --- receives ---> many Notifications
Incident 1 --- referenced by ---> many Notifications
```

MTTR and lifecycle data

Each Incident has startedAt and resolvedAt. Its mttrMinutes virtual is the rounded minute difference between them. The stats endpoint recomputes average MTTR by loading all incidents, selecting resolved records, and averaging in application memory.

Seed behavior

backend/scripts/seed.js deletes all three collections, creates four sample users, and inserts sample incidents/timeline/action-item data. This makes classroom setup fast but is destructive and has no environment safety guard.

7. End-to-End Request Flow

Login flow

```
1. User submits the Login form
2. AuthContext POSTs { email, password } to /api/auth/login
3. Express finds the User and bcrypt-compares the password
4. API returns JWT + public user data
5. Browser stores rcc_token and private routes render
6. Later API calls include Authorization: Bearer <token>
```

Incident flow

```
1. Authenticated user creates or edits an incident
2. Controller persists it and appends timeline events
3. Assignment / mentions create Notification documents
4. Dashboard/detail views reload REST data from MongoDB
5. Resolution sets resolvedAt; stats calculate MTTR
```

8. Findings and Recommendations

Blocker - Current database has no accounts

The local API was healthy, but the default database had zero User documents during this review. The displayed seed login fails because the account does not exist. Seed the demo database only when it is safe to reset it, or create/migrate real accounts.

Evidence: Live API check plus read-only MongoDB query; backend/scripts/seed.js creates the documented account.

High - Destructive seed script has no production safeguard

The seed command deletes Users, Incidents, and Notifications before inserting samples. A mistaken execution against a production URI would erase core data. Require an explicit DEMO_SEED=true flag, refuse NODE_ENV=production, and provide a backup/restore workflow.

Evidence: backend/scripts/seed.js, initial Promise.all deleteMany calls.

High - Insecure authentication defaults and demo credentials

JWT signing falls back to dev_secret if JWT_SECRET is unset; the UI embeds a simple, public seed password. Fail startup when a production secret is missing, remove credential defaults from deployed UI, enforce stronger passwords, and add password-reset/session management before production.

Evidence: backend/src/controllers/authController.js and backend/src/middleware/auth.js JWT fallbacks; frontend/src/pages/Login.jsx.

High - Incident edit state does not normalize populated assignment data

The edit form copies a fully populated Incident into local state, but its assignment select expects an ID string. Existing assignments can render incorrectly and the PATCH can submit an object where the API expects an ID. Build a dedicated editable DTO and normalize assignedTo to assignedTo?._id or assignedTo?.id before rendering or saving.

Evidence: frontend/src/components/IncidentForm.jsx form initialization and assignedTo select.

High - Environment template can break a new local install

frontend/.env.example uses http://<your-ec2-ip>:5001/api while the local README expects localhost. The installer copies this placeholder into frontend/.env, so the frontend can point to an invalid host until manually edited. Split local and production templates or default the sample to localhost.

Evidence: frontend/.env.example; installer.sh environment-file copy; README local setup.

High - Search accepts raw regular expressions

The incident q query is passed directly to new RegExp. Invalid syntax can throw and expensive patterns can consume server resources. Escape user input, impose query limits, or use a text index/search service; this is especially important while async errors are not consistently forwarded.

Evidence: backend/src/controllers/incidentController.js getIncidents search filter.

Medium - No login abuse protection or security hardening middleware

The API has no rate limiting, account lockout, Helmet/security headers, HTTPS enforcement, or explicit production cookie/session policy. Add express-rate-limit on auth routes, Helmet, TLS/reverse-proxy guidance, and structured production error responses.

Evidence: backend/package.json and backend/src/server.js.

Medium - Development CORS is broader than an allowlist

Outside production mode, any origin using port 5173 is accepted with credentials. A rejected origin reaches the generic error handler as a 500 rather than a clear CORS denial. Restrict development origins to known local hosts or explicit configuration and return an intentional 403 response.

Evidence: backend/src/server.js CORS origin callback and global error handler.

Medium - Browser token storage increases XSS blast radius

A JWT in localStorage can be read by JavaScript running on the origin. Use short-lived httpOnly secure cookies with CSRF protection, or document and mitigate the XSS risk with strict CSP and output-safety controls.

Evidence: frontend/src/context/AuthContext.jsx and frontend/src/services/api.js.

Medium - Input and relationship validation are incomplete

Controllers validate only a few required fields. Invalid ObjectIds, nonexistent assignees, weak passwords, oversized text, and unexpected values can lead to generic errors or inconsistent references. Add request-schema validation, ObjectId checks, length limits, and user existence checks.

Evidence: backend/src/controllers/authController.js and incidentController.js.

Medium - Incident permissions and reference cleanup need policy

Every authenticated user can currently edit every incident. Deleting users can leave reportedBy/timeline references, deleting incidents leaves notifications, and multi-document workflows lack transaction/cleanup rules. Define incident roles and implement validation plus explicit cascade or retention behavior.

Evidence: backend/src/routes/incidentRoutes.js, authController.js deleteUser, incidentController.js deleteIncident.

Medium - Async controller failures are not consistently forwarded

The controllers are async functions without a shared async-error wrapper or local try/catch. Under Express 4, rejected promises can bypass the final error middleware, leaving requests unresolved or producing unhandled-rejection behavior. Wrap controllers or upgrade/use an error-forwarding mechanism.

Evidence: backend/src/controllers/.js and backend/src/server.js error middleware.*

Medium - Reminder job can create repeat notification noise

Every stale high/critical incident is eligible again after the 30-minute threshold. The job stores no last-reminded marker or deduplication key, so it may repeatedly notify users and grow timelines/notifications indefinitely. Record reminder timestamps and throttle per incident/channel.

Evidence: backend/src/utils/reminderJob.js.

Medium - Global notifications cannot be marked read

The notification list includes records with targetUser set to null, but the mark-read query only updates notifications owned by the current user. An unassigned stale incident produces a global reminder that may stay unread for every viewer. Define broadcast read semantics or create per-user notifications.

Evidence: backend/src/controllers/notificationController.js and backend/src/utils/reminderJob.js.

Medium - No automated tests or CI quality gate

There are no test files or test/lint scripts in the repository. Add API integration tests for auth/RBAC/incidents, component tests for critical UI flows, and a CI job that runs build, lint, tests, and dependency checks.

Evidence: Root, frontend, and backend package.json scripts; repository inventory.

Low - Scaling limits in list/statistics/mention logic

The dashboard loads full incident/user collections, statistics are calculated in memory, search builds raw regular expressions from user input, and mention lookup reads all users for each comment. Pagination, indexes, aggregation, escaped search, and targeted lookup will be needed as data grows.

Evidence: backend/src/controllers/incidentController.js; frontend/src/pages/Dashboard.jsx.

Low - Accessibility and UI resilience need a pass

Keyboard focus is not visibly styled, the creation modal lacks full dialog focus behavior, drag/drop has no keyboard path, clickable notification rows are non-semantic, and most page loads lack error/empty states. Add accessible controls, focus-visible styles, dialog handling, and shared request-state UI.

Evidence: frontend/src/styles.css, IncidentForm.jsx, Notifications.jsx, Dashboard.jsx, and page load functions.

Low - Deployment and dependency setup are development-oriented

The backend does not serve the built frontend; scripts use Vite development/preview rather than a production hosting setup. There is no CI, process manager, container/reverse-proxy config, engines declaration, or pinned Vite/plugin version. Add a reproducible production pipeline using npm ci, supported Node versions, HTTPS/static hosting, and process supervision.

Evidence: Root/frontend package manifests, frontend package lock, backend/src/server.js, installer.sh, and README.

Low - Observability and reliability checks are basic

The health endpoint reports process availability but does not distinguish MongoDB readiness after startup, and raw internal error messages are returned by the generic error handler. Add readiness checks, connection event handling, normalized error payloads, structured logs, metrics, and graceful shutdown.

Evidence: backend/src/server.js and backend/src/config/db.js.

9. Recommended Delivery Plan

Immediate: make demos safe and reliable

- Restore/seed only the intended demo database, then verify login and all sample flows.
- Use a real JWT_SECRET in every environment and remove default credentials from non-demo builds.
- Fix frontend environment templates so a new local setup uses localhost by default.
- Guard the seed command against production and document backups.
- Normalize incident edit form values so existing assignments remain selected and PATCH payloads contain IDs, not populated objects.

Next sprint: application quality

- Add request validation, async error forwarding, error/loading/UI states, rate limiting, Helmet, and password policy.
- Add focused integration tests for login, role boundaries, incident mutation, and notifications.

- Improve accessibility: keyboard board controls, modal focus handling, semantic notification controls, and form autocomplete/type attributes.

Before production: operations and scale

- Adopt HTTPS behind a reverse proxy/static host, secure cookie sessions or a documented token strategy, production CORS allowlists, and secrets management.
- Add health/readiness metrics, structured logs, monitoring, database backup/restore, migration strategy, and a process manager/container deployment.
- Add pagination, database indexes, aggregation-backed statistics, reminder deduplication, and data-retention rules.
- Define notification ownership/broadcast semantics and use a distributed worker/lock if multiple API instances will run.

10. Running the Project Safely

For a local classroom/demo environment, configure MongoDB first and create environment files with local values. Use a unique, non-default JWT secret even for shared demos.

```
# From the repository root
npm run install:all
# Optional, destructive demo reset only:
npm run seed
npm run dev

# Frontend: http://localhost:5173
# Health:    http://localhost:5001/api/health
```

Configuration: Backend: set MONGODB_URI, JWT_SECRET, CLIENT_URL, and PORT. Frontend: set VITE_API_URL to the API base URL. Never copy the EC2 placeholder URL into a local environment unchanged.

Appendix A. Key Source Map

- frontend/src/main.jsx - application providers and browser routing bootstrap.
- frontend/src/App.jsx - public/private routing.
- frontend/src/context/AuthContext.jsx and services/api.js - token/session and HTTP client.
- frontend/src/pages - Login, Dashboard, IncidentDetail, Team user flows.
- backend/src/server.js - Express, CORS, health endpoint, startup.
- backend/src/middleware/auth.js - token verification and admin authorization.
- backend/src/controllers - business logic for auth, incidents, and notifications.
- backend/src/models - User, Incident, and Notification Mongoose schemas.
- backend/src/utils/reminderJob.js - periodic stale-incident notification logic.

- backend/scripts/seed.js - destructive demo-data reset and sample credentials.

Appendix B. Audit Boundaries

This report is based on repository source inspection, local API/build observations, and configuration/script review. It is not a network penetration test, a dependency-vulnerability scan, a load test, or a review of external infrastructure, production logs, cloud permissions, DNS, TLS certificates, or MongoDB server hardening. Those should be assessed before a real production launch.